# most common engine failures 2000-2020

google.com/search

most common engine failures 2000-2020

15 March 2026

During the period 2000–2020, engine failures were frequently caused by a shift toward more complex, high-efficiency technologies—such as turbocharging, direct injection, and cylinder deactivation—which introduced new points of vulnerability

. YouTube +1

The most common general failure types across this era included:

- **Lubrication and Oil Sludge:** Improper oil management or design flaws led to "oil sludge," which blocked flow and caused catastrophic failures.
- **Timing System Failures:** Both timing chains and belts were major failure points, often due to faulty hydraulic tensioners or "wet belt" designs that disintegrated into the oil system.
- **Cooling System Defects:** Overheating from water pump failures or poorly managed turbocharger heat often resulted in cracked cylinder heads or blown head gaskets.
- **Cylinder Deactivation Issues:** Systems like GM's Active Fuel Management (AFM) were notorious for lifter failures that subsequently damaged camshafts. Reddit +8

Infamous Engine Failures (2000–2020)

| Engine / Manufacturer | Era | Key Failure Modes |
|---|---|---|
| **Hyundai/Kia Theta II (2.0L/2.4L GDI)** | 2010–2020 | Manufacturing debris restricted oil flow, leading to **rod bearing failures, seizures, and fires**. |

| | | |
|---|---|---|
| **Volkswagen 1.4/1.6 FSI & 1.2 TSI** | 2001–2015 | Prone to **timing chain stretching** and tensioner failure, causing severe internal damage. |
| **BMW N63 (4.4L Biturbo V8)** | 2008–2012 | Excessive heat from turbochargers caused **leaking injectors and timing chain wear**. |
| **Toyota 1.8L VVT-i (1ZZ-FE)** | 1999–2007 | Flawed oil piston rings led to **extreme oil consumption** and eventual engine failure. |
| **Mazda 1.3L Renesis (Rotary)** | 2004–2012 | Wear of **apex seals** caused loss of compression and frequent engine rebuild requirements. |
| **Subaru 2.5L (EJ25)** | 1996–2011 | Notorious for **head gasket failures** and overheating issues. |
| **Ford EcoBoost (1.6L)** | 2011–2020 | Early versions suffered from **coolant leaks and overheating**, which posed fire risks. |
| **Nissan 1.5L/1.8L (QG Series)** | 1999–2006 | High oil consumption (over 1.5L per 1000km) due to **premature piston ring wear**. |

Summary of Component-Specific Failures

- **Ignition Systems:** Faulty coil packs and aged spark plugs were leading causes of misfires and poor fuel economy in modern engines.
- **Sensors:** Malfunctioning Mass Airflow (MAF) and oxygen sensors frequently caused rough idling and reduced efficiency.
- **Fuel Injectors:** Clogging from soot and debris disrupted precise fuel delivery, especially in direct-injection models. ASM Auto Recycling +3

Are you looking for reliability data on a **specific car model** or more information on **maintaining** these particular engines?

AI responses may include mistakes. Learn more

19 sites

- What Are Some Reliable Engines from the Last 10-15 Years ...

  The older Mazda FS, F2, and FE8v were workhorses as well. They had some issues with the outsourced automatic transmissions, but th...

  Reddit

- The WORST Engines EVER MADE Won't Last 60000 Miles ...

  Many engines, despite good maintenance, are susceptible to major breakdowns in as little as 60,000 miles. Poorly designed engines ...
  YouTube·ExoticCar PlayPlace

  2:44:07

- The 9 most common car engine problems revealed

  The 9 most common car engine problems revealed * Poor lubrication. It's absolutely vital that your engine receives enough oil betw...

  ASM Auto Recycling

Show all

before we had conversation on diagnosing engine faults based on 5-gas emissions readings enhanced with potential OBD DTC reading amd if available with PID reading - i want to know these correlations for stated most common engine faults as for example piston ring wear(excessive oil consumption) and others stated above - so first just 5-gas readings then DTC and finally PID and correlation of all factors
15 March 2026

To diagnose complex engine failures from the 2000–2020 era, combining

**5-gas emissions**, **DTCs**, and **PIDs** allows you to see the "cause and effect" chain. 5-gas readings show the raw chemical outcome of combustion, while OBD data reveals the engine management's attempt to compensate.

1. Piston Ring Wear (Excessive Oil Consumption)

This fault introduces liquid hydrocarbons (oil) into the combustion process, which do not burn as cleanly as gasoline.

- **5-Gas Readings:**
  - **HC (Hydrocarbons):** Significantly elevated (often >150-300 ppm at idle).
  - **O2 (Oxygen):** May fluctuate; if oil fouls the spark plug, O2 rises due to misfires.
  - **CO2 (Carbon Dioxide):** Slightly lower as combustion efficiency drops.
- **DTC (Diagnostic Trouble Codes):**
  - **P0300-P030X:** Random or cylinder-specific misfires due to oil-fouled plugs.
  - **P0420/P0430:** Catalyst Efficiency Below Threshold (burned oil poisons the catalytic converter).

- **PID (Parameter IDs):**
  - **Short Term Fuel Trim (STFT):** May show positive (adding fuel) if misfires cause high O2 readings.
  - **Crankcase Pressure:** High "blow-by" pressure if rings aren't sealing.

Open Access CAAS Agricultural Journals +4

2. Timing System Failure (Stretched Chain/Belt)

Common in VW TSI/FSI and early 2000s Nissan engines, this alters the mechanical "breathing" of the engine.

- **5-Gas Readings:**
  - **NOx (Nitrogen Oxides):** Can spike if timing is advanced or drop if retarded.
  - **CO (Carbon Monoxide):** Higher if the engine is struggling with volumetric efficiency.
  - **O2:** Often higher as valves are not closed during the full power stroke, allowing unburned air to pass through.
- **DTC:**

  **P0011/P0016/P0017:** Camshaft/Crankshaft Position Correlation codes.
- **PID:**
  - **Calculated Load:** Higher than normal at idle (engine is working harder to overcome timing issues).
  - **VVT Angle/Duty Cycle:** Maxed out or frozen as the ECU tries to compensate for the stretch.

3. Cooling System Defects (Head Gasket Failure)

Frequent in Subaru EJ25 and Ford 1.6L EcoBoost engines.

DFC Diesel

- **5-Gas Readings:**
  - **CO2:** Found in the radiator (using a block tester, which is essentially a 5-gas check of the coolant).
  - **HC:** May be low in the exhaust but present in the cooling system.
  - **O2:** High if coolant is "steaming" out the fire in a cylinder.
- **DTC:**
  - **P0300:** Misfire.
  - **P0117/P0118:** Engine Coolant Temp (ECT) sensor circuit low/high (due to air pockets/overheating).
- **PID:**
  - **ECT (Engine Coolant Temp):** Erratic readings or rapid spiking.
  - **Misfire Counts:** High counts on adjacent cylinders.

4. Cylinder Deactivation Issues (GM AFM/DOD)

Failures usually involve mechanical lifter collapse.

- **5-Gas Readings:**
  - **HC:** Spike when the deactivated cylinder is supposed to be "off" but is still receiving/leaking fuel or oil.
  - **O2:** High (lean) if the valve remains closed and the ECU isn't fueling correctly.
- **DTC:**
  - **P0300:** Misfire.
  - **P3400/P3401:** Cylinder Deactivation System performance.
- **PID:**
  - **Fuel Trim (STFT/LTFT):** Significant bank-to-bank imbalance.
  - **MAP (Manifold Absolute Pressure):** Higher than normal due to reduced engine vacuum from the "dead" cylinder.

Correlation Summary Table

| Fault | 5-Gas Key Indicator | Primary DTC | Critical PID |
|---|---|---|---|
| **Ring Wear** | High HC, Low CO2 | P0420, P0300 | High Crankcase Pressure |
| **Timing Stretch** | High O2, Low NOx | P0016 / P0017 | VVT Cam Angle Error |
| **Head Gasket** | CO2 in Radiator | P030X, P0128 | Spiking ECT |
| **Lifter Failure** | High HC/O2 | P3400, P0300 | MAP Sensor (Low Vacuum) |

Would you like to dive deeper into the **PID graphing patterns** for a specific manufacturer like **Hyundai** or **BMW**?

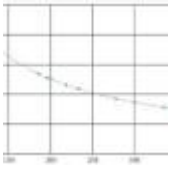
AI responses may include mistakes. Learn more

8 sites

- Monitoring of selected emissions of internal combustion engine

  The combustion of hydrocarbon fuels (diesel) emissions result of various kinds. The gaseous pol- lutants from diesel engines mainl...

  Open Access CAAS Agricultural Journals



- Impact of Blow-By Gas and Endgap Ring Position on the Variations ...

  Wroblewski and Koszalka [18] measured the influence of various anti-wear coatings on frictional losses on the rings. Wroblewski an...
  PSE Community.org

- Garage Gurus | How to Use a Five-Gas Analyzer to Diagnose ...

  A five-gas analyzer can be used as an extra tool to help diagnose a vehicle properly and avoid costly comebacks. To use the analyz...
  YouTube·Garage Gurus

  3m

Show all

lets do similar research for another for 10 more known faults for cars of this era

15 March 2026

Building on the diagnostic framework of combining chemical outputs with electronic data, here are 10 additional common engine failures from the 2000–2020 era. These correlations help bridge the gap between "what the exhaust says" and "what the computer sees."

1. Carbon Buildup on Intake Valves (Direct Injection)

Common in **BMW N54/N55**, **VW/Audi 2.0T**, and **GM 3.6L**. Since fuel doesn't wash the valves in GDI engines, soot accumulates and restricts airflow.

- **5-Gas Readings:**
  - **HC:** High at idle due to "tumble" interference causing incomplete combustion.
  - **O2:** High (Lean) because the restriction prevents a full air charge, often leading to misfires.
- **DTC:**
  **P0300/P030X:** Random misfires, especially on cold starts.
- **PID:**
  - **STFT:** Highly positive (adding fuel) to compensate for perceived lean conditions.
  - **Mass Air Flow (MAF):** Lower than specified at wide-open throttle (WOT) due to intake restriction.

2. Variable Cam Timing (VCT) Phaser Failure

Infamous in the **Ford 5.4L Triton** and **GM High Feature V6**. Phasers fail to lock or move smoothly due to low oil pressure or internal wear.

- **5-Gas Readings:**
  - **NOx:** High if the cam is stuck in an advanced position (increased cylinder pressure).
  - **CO:** Elevated if the timing retardation prevents efficient burning at high loads.

- **DTC:**
  - **P0011/P0021:** Intake Camshaft Position Timing Over-Advanced.
  - **P0340/P0345:** Camshaft Position Sensor Circuit Malfunction.
- **PID:**
  - **VCT_ERR (Variable Cam Timing Error):** Shows a large discrepancy between commanded and actual cam angle.
  - **Oil Pressure:** Often lower than 15-20 PSI at hot idle (a root cause).

3. High-Pressure Fuel Pump (HPFP) Failure

A staple issue for early **BMW (N54)** and **VW (2.0T)** models. The mechanical pump fails to reach the several thousand PSI required for GDI.

- **5-Gas Readings:**
  - **HC:** Extremely high during cranking or hard acceleration (poor atomization).
  - **O2:** High (Lean) as the engine starves for fuel.
- **DTC:**
  **P0087:** Fuel Rail/System Pressure - Too Low.
- **PID:**
  **Fuel Rail Pressure (Actual vs. Desired):** Actual pressure will stay near the low-pressure pump (~70-100 PSI) instead of rising to 1,500+ PSI.

4. Turbocharger Wastegate/Boost Leaks

Common in **BMW N55** and **Ford EcoBoost** engines. Plastic "charge pipes" crack or wastegate actuators rattle and fail to seal.

- **5-Gas Readings:**
  - **CO:** Very high under load (engine runs rich as it expects boost that isn't there).
  - **CO2:** Low under load because the engine isn't reaching peak efficiency.
- **DTC:**
  **P0299:** Turbocharger Underboost.
- **PID:**
  - **Boost Pressure (MAP):** Lower than "Commanded Boost" under load.
  - **Wastegate Duty Cycle:** Maxed out at 95-100% as the ECU tries to force boost.

5. PCV System/Oil Separator Failure

Frequent in **BMW (Valve Cover)** and **Audi/VW (PCV Diaphragm)**. A torn diaphragm allows unmetered air and oil vapor into the intake.

- **5-Gas Readings:**
  - **HC:** Spikes if oil is sucked into the intake.
  - **O2:** High (Lean) due to the vacuum leak caused by the torn diaphragm.
- **DTC:**
  **P0171/P0174:** System Too Lean (Bank 1 or 2).

- **PID:**
  **LTFT (Long Term Fuel Trim):** High positive (e.g., +15-25%) at idle, but improves at higher RPMs.

6. Clogged Catalytic Converter

Often a "downstream" failure from oil burning or misfires.

- **5-Gas Readings:**
  - **CO/HC:** Very low (the cat is still "working" but the car has no power) or high if it has melted internally.
  - **O2:** Low (the cat is "bottling" the exhaust).
- **DTC:**
  **P0420:** Catalyst Efficiency Below Threshold.
- **PID:**
  - **Downstream O2 Sensor (Sensor 2):** Mimics the upstream sensor (oscillating rapidly) instead of staying flat, indicating no oxygen storage.
  - **MAP Sensor:** Shows high intake manifold pressure (low vacuum) at idle because the engine can't "exhale."

7. Evaporative Emission (EVAP) Purge Valve Stuck Open

Common across most brands (GM, Ford, Toyota). The valve allows raw fuel vapors to flood the intake.

- **5-Gas Readings:**
  - **CO:** High at idle (engine is running rich from unmetered fuel vapors).
  - **CO2:** Lower than normal.
- **DTC:**
  **P0441/P0442:** Incorrect Purge Flow or Small Leak.
- **PID:**
  **STFT:** Highly negative (e.g., -20%) at idle as the ECU tries to cut fuel to compensate for the vapor.

8. Exhaust Gas Recirculation (EGR) Stuck Open

A classic failure in **Toyota (Prius/Corolla)** and **Ford** engines, leading to rough idling or stalling.

- **5-Gas Readings:**
  - **NOx:** Extremely low (EGR is active when it shouldn't be).
  - **O2:** High if the engine misfires from the dilution.
- **DTC:**
  **P0401:** EGR Flow Insufficient (if clogged) or **P0402** (Excessive flow).
- **PID:**
  **MAP Sensor:** Reading closer to atmospheric pressure at idle (low vacuum) because the intake is being filled with exhaust gas.

9. Mass Air Flow (MAF) Sensor Contamination

Common in vehicles with "oiled" aftermarket filters or high mileage (dust buildup).

- **5-Gas Readings:**
  - **CO:** Low (Lean) during acceleration because the MAF under-reports air intake.
  - **O2:** High during acceleration.
- **DTC:**
  **P0101:** MAF Circuit Range/Performance.
- **PID:**
  **MAF (g/s):** Lower than the "1.0g/s per Liter of Displacement" rule of thumb at idle (e.g., a 3.0L engine showing only 2.1g/s).

10. Cooling Fan/Thermostat Failure (Overheating)

Frequent in **BMW (Electric Water Pumps)** and **Chrysler (Pentastar)** models.

- **5-Gas Readings:**
  - **NOx:** Spikes significantly as cylinder temperatures rise.
  - **CO2:** May drop if pre-ignition (knock) begins to occur.
- **DTC:**
  **P0128:** Coolant Thermostat (Coolant Temp Below Regulating Temp).
- **PID:**
  **ECT (Engine Coolant Temp):** Rising above 220°F (105°C) without the cooling fan duty cycle increasing.

Correlation Summary: 5-Gas vs. PID Trends

| Fault | 5-Gas "Fingerprint" | PID "Telltale" |
|---|---|---|
| **Carbon Buildup** | High HC / High O2 | High positive STFT at idle |
| **VCT Phaser** | High NOx (if advanced) | VCT Error > 5 degrees |
| **Boost Leak** | Very High CO (under load) | MAP < Commanded Boost |
| **Stuck Purge** | High CO (at idle) | Highly Negative STFT (-20%+) |
| **Bad MAF** | High O2 (under load) | Low g/s vs. Displacement |

Would you like to analyze a specific **real-world diagnostic case** where you have these readings, or should we look at the **graphical wave patterns** of these sensors?

AI responses may include mistakes. Learn more

8 sites

- TOP 10 MOST COMMON DTCs - ALLDATA

  1) P0300 - Random/Multiple Cylinder Misfire: Indicates that the engine is misfiring in multiple cylinders or the misfire is occurr...

  www.alldata.com

- 10 Most Frequent Diagnostic Trouble Codes (DTCs) and What ...

  Common Reasons: Faulty spark plugs or ignition coils. Vacuum leaks. Fuel delivery issues (e.g., clogged injectors or fuel pump pro...

  WrenchLane

- Diagnosing a Ford 5.4 Cam Timing Issue — Before You Tear It ...

  A variable cam timing advance of 39° is observed on bank one, accompanied by a VCT error of -39°. This offset suggests that the co...
  YouTube·ScannerDanner

  44:57

Show all

i am building simple web app in which will user input 5-gas readings and also potential DTC or PID if available and app will return possible culprits - but i do not know how to approach building database which will enable diagnostic - i guess there will be need for tables which will crossreference known patterns?
15 March 2026

To build a diagnostic engine, you shouldn't just look for "exact matches." Engine data is "fuzzy"—a vacuum leak might show an STFT of +15% on one car and +22% on another.

The most robust approach is a **Weighted Scoring System** (or a simplified Heuristic Engine). You need a database that maps "Symptoms" to "Culprits" with a confidence weight.

1. The Database Schema (Relational Approach)

You need four primary tables to make the cross-referencing work:

- `Culprits` **Table:** (e.g., "Piston Ring Wear", "Vacuum Leak", "Stuck Open Injector").
- `Symptoms` **Table:** Individual data points (e.g., "High HC", "Low CO2", "P0171", "Negative STFT").
- `Diagnostic_Logic` **Table:** This is your "Cross-reference." It links a Culprit to many Symptoms.
  *Columns:* `culprit_id`, `symptom_id`, `weight` (1–10), `requirement_type` (Is this symptom "Mandatory", "Possible", or "Exclusionary"?).
- `Thresholds` **Table:** To translate raw user input into "Symptoms."
  *Example:* If user inputs `HC = 400`, the app looks here and flags the "High HC" symptom.

2. The Logic Flow

Your app shouldn't just "query" the database; it should "score" it:

1. **Normalization:** Convert user inputs into states.
   - *Input:* CO: 4.5% → *State:* `CO_VERY_HIGH`.
   - *Input:* STFT: -18% → *State:* `STFT_RICH_CORRECTION`.
2. **Matching:** Find all Culprits in the `Diagnostic_Logic` table that share those states.
3. **Scoring:** Sum the `weights`.
   - If "Stuck Open Purge Valve" has `CO_VERY_HIGH` (Weight 8) and `STFT_RICH_CORRECTION` (Weight 9), its total score is 17.
   - If "Clogged Air Filter" only matches `CO_VERY_HIGH` (Weight 5), its score is 5.
4. **Ranking:** Return the Culprits with the highest scores to the user.

3. Example Logic Entry: "Vacuum Leak"

In your database, the "Vacuum Leak" culprit would be linked to:

| Category | Data Point | Condition | Weight |
|---|---|---|---|
| **5-Gas** | O2 | High (> 2.0%) | 8 |
| **5-Gas** | CO2 | Low (< 12%) | 5 |
| **DTC** | P0171/P0174 | Present | 10 |
| **PID** | STFT | High Positive (> 10%) | 9 |
| **PID** | MAP | Low Vacuum (High kPa) | 7 |

4. Advanced: "Conditional" Correlation

To make the app professional, add a **"Delta" logic**.

- *Example:* "If HC is High **AND** O2 is High, increase weight for 'Ignition Misfire'."
- *Example:* "If STFT is High at Idle but Low at 2500 RPM, increase weight for 'Vacuum Leak'."

5. Implementation Strategy

- **Backend:** Use a **JSON-based NoSQL** (like MongoDB) or a **PostgreSQL** database. JSON is great because different engines (GDI vs. Port Injection) have different "normal" ranges, and JSON handles that flexibility well.
- **Frontend:** Simple forms for the 5 gases, a multi-select for common DTCs, and a "Common PIDs" section.

Would you like me to write a **sample JSON structure** for one of these faults so you can see how it would look in a database?

AI responses may include mistakes. Learn more

yes please

15 March 2026

To build a diagnostic engine that correlates 5-gas readings with OBD-II data, you need a structured data model that allows for "fuzzy" matching

. A **weighted symptom-to-culprit** approach is more effective than exact matches because sensor values fluctuate based on engine load and wear. Medium +1

Below is a sample JSON structure for a single fault ("Piston Ring Wear") that you can use to populate your database. Stack Overflow +1

Sample JSON Structure: "Piston Ring Wear"

json

```
{
  "culprit_id": "FLT_001",
  "name": "Piston Ring Wear (Excessive Oil Consumption)",
  "description": "Wear in the compression or oil control rings allowing oil into
combustion chamber and blow-by into crankcase.",
  "confidence_threshold": 65,
  "logic_rules": {
    "five_gas": [
      { "gas": "HC", "condition": "high", "min": 200, "unit": "ppm", "weight": 9,
"importance": "critical" },
      { "gas": "O2", "condition": "high", "min": 1.5, "unit": "%", "weight": 5,
"importance": "supporting" },
      { "gas": "CO2", "condition": "low", "max": 12.5, "unit": "%", "weight": 4,
"importance": "supporting" }
    ],
    "dtc_codes": [
      { "code": "P0300", "description": "Random Misfire", "weight": 8 },
      { "code": "P0420", "description": "Catalyst Efficiency Low", "weight": 10 }
    ],
    "pids": [
      {
        "pid_name": "Short Term Fuel Trim",
        "mode": "01",
        "pid": "06",
        "condition": "positive",
        "min": 10,
        "weight": 7
      },
      {
        "pid_name": "Manifold Absolute Pressure",
        "mode": "01",
        "pid": "0B",
        "condition": "high_at_idle",
        "weight": 6
      }
    ]
  },
  "user_guidance": "Check for blue smoke on acceleration and perform a dry/wet
compression test."
}
```

Use code with caution.

Key Data Components for Your App

To make this searchable and functional, your database should organize these elements:

- **Weights (1-10):** Not all symptoms are equal. A `P0420` (Catalyst Efficiency) code is a stronger indicator of oil burning than a slightly low `CO2` reading.
- **Logical Operands:** Use "AND/OR" logic. For example, `High HC` **AND** `High O2` strongly suggests a misfire rather than just a rich condition.

- **Normalized States:** Your app should translate raw inputs (e.g., `HC: 450`) into states like `HC_EXTREMELY_HIGH` before running the scoring engine.
- **Importance Levels:** Mark some readings as "Critical" (must be present for this diagnosis) versus "Supporting" (increases confidence if present).

Would you like a **Python snippet** to see how to calculate a "Confidence Score" using this JSON data?

AI responses may include mistakes. Learn more

12 sites

- How to store and process rules and logic with JsonLogic

  Structure. Everything is explained a lot better on the home page but I will try to go on with what I want to point out. There are ...

  Medium

- OBD2 Explained - A Simple Intro [2025] – CSS Electronics

  OBD2 Explained - A Simple Intro [2025] * Need a simple, practical intro to OBD2? In this guide we introduce the On Board Diagnosti...
  CSS Electronics·CSS Electronics

  10:37
- Sample JSON messages for Engine Diagnostic data

  Currently the web server returns an HTML formatted table with the diagnostic data and a JSON string with the same data: { "Raw Wat...
  Google Groups

Show all

Yes

15 March 2026

To implement the diagnostic engine in your web app, you can use a

**weighted scoring algorithm**. This approach compares user inputs against your JSON fault models and calculates a "Confidence Score" by dividing the total weight of matched symptoms by the maximum possible weight for that specific fault.

Python Diagnostic Engine Snippet

The following function demonstrates how to process 5-gas readings, DTCs, and PIDs to produce a percentage-based confidence level:

python

```
def calculate_confidence(user_input, fault_model):
    score = 0
    max_possible_score = 0

    # 1. Evaluate 5-Gas Readings
    for gas_rule in fault_model['logic_rules']['five_gas']:
        max_possible_score += gas_rule['weight']
        val = user_input['five_gas'].get(gas_rule['gas'])
        if val is not None:
            # Check if gas reading meets the 'high' or 'low' threshold
            if gas_rule['condition'] == 'high' and val >= gas_rule['min']:
                score += gas_rule['weight']
            elif gas_rule['condition'] == 'low' and val <= gas_rule['max']:
                score += gas_rule['weight']

    # 2. Evaluate DTCs
    for dtc_rule in fault_model['logic_rules']['dtc_codes']:
        max_possible_score += dtc_rule['weight']
        if dtc_rule['code'] in user_input['dtcs']:
            score += dtc_rule['weight']

    # 3. Evaluate PIDs
    for pid_rule in fault_model['logic_rules']['pids']:
        max_possible_score += pid_rule['weight']
        pid_val = user_input['pids'].get(pid_rule['pid_name'])
        if pid_val:
            if pid_rule['condition'] == 'positive' and pid_val > 0:
                score += pid_rule['weight']
            elif pid_rule['condition'] == 'high_at_idle' and pid_val > 35:
                score += pid_rule['weight']

    # Calculate final percentage
    confidence = (score / max_possible_score) * 100 if max_possible_score > 0 else 0
    return round(confidence, 2)
```

Use code with caution.

Integration Tips for Your Web App

- **Normalize Your Inputs**: Before running the calculation, ensure your web app converts various units (like Lambda vs. Air-Fuel Ratio) into a consistent format.
- **Iterate Through All Faults**: Loop this function through every `fault_model` in your database and return the top 3-5 results with the highest confidence scores to the user.
- **Confidence Thresholds**: You might want to hide results that fall below a certain confidence level (e.g., < 30%) to reduce "noise" in the diagnosis.

LobeHub +1

Would you like me to generate a **structured list of thresholds** (min/max values) for the other 10 common faults to help fill out your database?

AI responses may include mistakes. Learn more

2 sites

- n8n-automation | Skills Marketplace - LobeHub

  Phase 2-3: Rule-Based Orchestrator * Implement agent_router function (Python/FastAPI endpoint) * Route based on: alert type/severi...

  LobeHub

- Building a Python-Powered, AI-Enabled SOC Playbook

  Python Script for JSON Log Normalization: import json. from datetime import datetime def normalize(log): try: ts = datetime.strpti...

  Medium

yes

15 March 2026

To help you populate your database, here is a structured list of thresholds for the 10 common faults discussed earlier. You can map these directly into your JSON

`logic_rules`.

Diagnostic Thresholds for Database Mapping

| Fault Name | 5-Gas Key Thresholds | Primary DTCs | Key PID Thresholds |
|---|---|---|---|
| **1. Intake Carbon** | HC > 80ppm, O2 > 1.5% | P0300, P0301-4 | STFT > +10% (Idle), MAF < 80% of Spec (WOT) |
| **2. VCT Phaser** | NOx > 600ppm (if adv.) | P0011, P0021 | VCT_ERR > 5°, Oil Press < 20 PSI (Hot Idle) |
| **3. HPFP Failure** | HC > 500ppm (Crank) | P0087 | Fuel Rail Press < 200 PSI (Cmd: 1500+) |
| **4. Boost Leak** | CO > 3.0% (Under Load) | P0299 | MAP < (Baro + 5 psi) at WOT |
| **5. PCV Failure** | O2 > 3.0%, HC > 100ppm | P0171, P0174 | LTFT > +15% (Idle), improves at 2500 RPM |
| **6. Clogged Cat** | CO/HC Low (Stored) | P0420 | Downstream O2 switching > 0.5Hz |

| | | | |
|---|---|---|---|
| **7. Stuck Purge** | CO > 2.0% (Idle) | P0441, P0442 | STFT < -15% (Idle) |
| **8. EGR Stuck Open** | NOx < 10ppm, O2 High | P0402 | MAP > 45 kPa at Idle (Low Vacuum) |
| **9. Dirty MAF** | O2 > 2.0% (Load) | P0101, P0171 | MAF < 1.0 g/s per Liter of Disp. (Idle) |
| **10. Overheating** | NOx > 1000ppm | P0128, P0117 | ECT > 225°F (107°C) |

Implementation Logic for your App:

1. **Load-Dependent Rules:** Note that some faults (like Boost Leaks or Dirty MAF) require readings **under load** (2500+ RPM or WOT). Your UI should ask the user: *"Is this reading at Idle or 2500 RPM?"*
2. **The "Lambda" Factor:** If your 5-gas analyzer provides **Lambda**, use it as a master filter. `Lambda > 1.05` is Lean (Weights toward Vacuum Leaks/MAF); `Lambda < 0.95` is Rich (Weights toward Purge/Injectors).
3. **DTC Multiplier:** In your code, give DTCs a **"Global Multiplier"** (e.g., `1.5x`). If a user enters a DTC that matches a fault, it should significantly "pull" that fault to the top of the list compared to gas readings alone.

Next Step for Your App:

Would you like a **sample HTML/JavaScript form structure** that handles these multi-input fields (Gas, DTC, PID) and sends them to your Python backend?

AI responses may include mistakes. Learn more

Yes great!

15 March 2026

To build a functional diagnostic tool, your frontend needs to capture three distinct types of data and send them to your Python backend as a single JSON object.

The following structure uses the Fetch API to send data to a Flask or Django endpoint.

1. HTML Structure (`index.html`)

This form uses `fieldsets` to group the 5-gas, DTC, and PID inputs logically.

html

```
<form id="diagnosticForm">
  <!-- 5-Gas Readings -->
  <fieldset>
    <legend>5-Gas Analysis</legend>
    <label>HC (ppm): <input type="number" name="hc" placeholder="e.g. 450"></label>
    <label>CO (%): <input type="number" step="0.1" name="co"></label>
    <label>O2 (%): <input type="number" step="0.1" name="o2"></label>
    <!-- Add CO2 and NOx as needed -->
  </fieldset>

  <!-- DTC Entry -->
  <fieldset>
    <legend>DTC Codes</legend>
    <input type="text" id="dtcInput" placeholder="Enter code (e.g. P0300)">
    <button type="button" onclick="addDTC()">Add Code</button>
    <ul id="dtcList"></ul>
  </fieldset>

  <!-- Key PIDs -->
  <fieldset>
    <legend>Critical PIDs</legend>
    <label>Short Term Fuel Trim (%): <input type="number" name="stft"></label>
    <label>MAP (kPa): <input type="number" name="map"></label>
  </fieldset>

  <button type="submit">Analyze Faults</button>
</form>

<div id="results"></div>
```

Use code with caution.

2. JavaScript Logic (`app.js`)

This script captures the form data and formats it into the JSON structure your Python `calculate_confidence` function expects.

javascript

```
let dtcs = [];

function addDTC() {
    const code = document.getElementById('dtcInput').value.toUpperCase();
    if (code && !dtcs.includes(code)) {
        dtcs.push(code);
        const li = document.createElement('li');
        li.textContent = code;
        document.getElementById('dtcList').appendChild(li);
        document.getElementById('dtcInput').value = '';
    }
}

document.getElementById('diagnosticForm').addEventListener('submit', async (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);

    // Construct the payload
    const payload = {
        five_gas: {
            HC: parseFloat(formData.get('hc')),
            CO: parseFloat(formData.get('co')),
            O2: parseFloat(formData.get('o2'))
        },
        dtcs: dtcs,
        pids: {
            "Short Term Fuel Trim": parseFloat(formData.get('stft')),
            "Manifold Absolute Pressure": parseFloat(formData.get('map'))
        }
    };

    // Send to Python Backend
    const response = await fetch('/analyze', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
    });

    const results = await response.json();
    document.getElementById('results').innerHTML = `<h3>Possible Culprit:
${results.top_match}</h3>`;
});
```

Use code with caution.

UI Design Inspiration

For a professional "shop tool" look, consider a dark-mode dashboard interface with real-time feedback.

[Dark Mode Dashboard – Real-Time Analytic by Dmitry Sergushkin ...](#)

[Dribbble](#)

[Dark Mode Dashboard designs, themes, templates and ...](#)

[Dribbble](#)

shutterstock.com · 1995175394

[10,575 Dark Dashboard Ui Stock Vectors and Vector Art | Shutterstock](#)

[Shutterstock](#)

[12,349 Dark Dashboard Ui Royalty-Free Images, Stock Photos ...](#)

[Shutterstock](#)

[Dark Dashboard designs, themes, templates and downloadable ...](#)

[Dribbble](#)

shutterstock.com · 1989216497

[15,541 Dashboard Dark Stock Vectors and Vector Art | Shutterstock](#)

[Shutterstock](#)

[Dark Mode Dashboard designs, themes, templates and ...]

[Dribbble]

[Dark Mode Dashboard Ui Modern Presentation Stock Vector ...]

[Shutterstock]

[Dashboard design in dark colors. App interface with UI and UX ...](#)

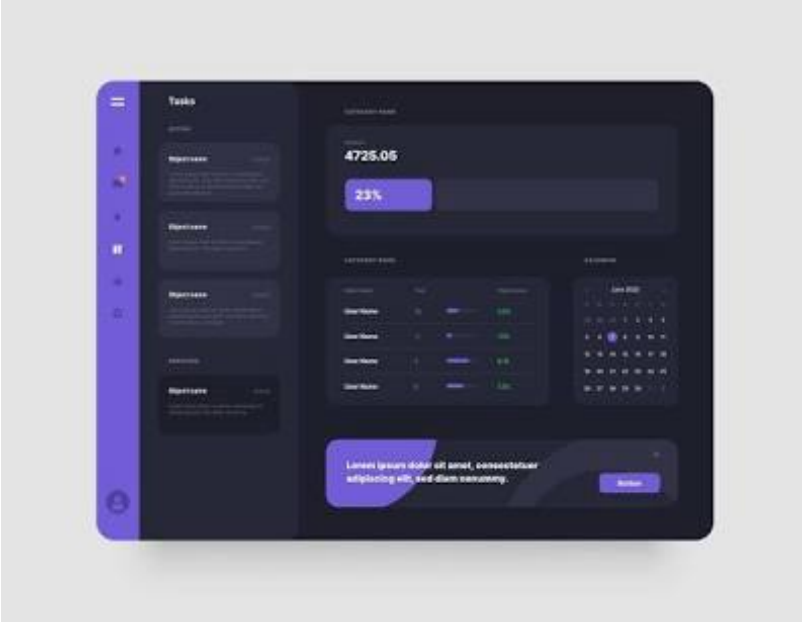
[Vecteezy](#)

[Browse thousands of Dark Mode Dashboard UI images for design ...](#)

[Dribbble](#)

[25 Dark Mode UI Design Examples | EASEOUT]

[EASEOUT]




[Dashboard Design. App interface with UI and UX elements. Use ...]

[Vecteezy]

[25 Dark Mode UI Design Examples | EASEOUT]

[EASEOUT]

[Analytics Dashboard Dark Mode - MingCute | Figma]

[Figma]

[60+ Best Dark mode screen 2026 UI/UX Inspiration | Muzli]

[Muzli]

[Dashboard Design. App interface with UI and UX elements. Use ...]

[Vecteezy]

[Dark Mode Dashboard Projects :: Photos, videos, logos, illustrations ...]

[Behance]

[Dashboard Design. App interface with UI and UX elements. Use ...]

[Adobe Stock]

[Modern Dashboard, Dashboard design trend 2021, Dashboard ...](#)

[Behance](#)

[Browse thousands of Dark Modern Dashboard images for design ...](#)

[Dribbble](#)

Pro-Tips for Web Implementation

- **Input Validation**: Add `min="0"` and `step="0.01"` to your HTML inputs to prevent users from entering invalid negative numbers or losing precision on small gas readings.
- **Loading States**: Since your Python engine might loop through dozens of fault models, use a "Calculating..." spinner on the button to improve user experience.
- **CSV Export**: Allow users to save their results as a PDF or CSV report for customer documentation.

Would you like to see how to set up the **Flask backend routes** to receive this data and return the final diagnosis?

AI responses may include mistakes. Learn more

Yes please!

15 March 2026

To connect your frontend to the diagnostic engine, you can use

**Flask**, a lightweight Python framework. Your backend will serve as the "brain," receiving data from the web form, running the weighted scoring logic, and returning the most likely engine faults as a JSON response.

1. Flask Backend Implementation (`app.py`)

This script sets up a POST route to receive data from your `fetch()` request and processes it against your database of fault models.

python

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock database - in a real app, load this from a JSON file or SQL DB
FAULT_MODELS = [
    {
        "name": "Piston Ring Wear",
        "user_guidance": "Blue smoke? Check crankcase pressure and do a wet compression
test.",
        "logic_rules": {
            "five_gas": [{"gas": "HC", "condition": "high", "min": 200, "weight": 10}],
            "dtc_codes": [{"code": "P0420", "weight": 8}, {"code": "P0300", "weight":
5}],
            "pids": [{"pid_name": "Short Term Fuel Trim", "condition": "positive",
"weight": 7}]
        }
    }
    # Add other 10+ models here...
]

@app.route('/analyze', methods=['POST'])
def analyze():
    user_data = request.json  # Data from your JavaScript fetch()
    results = []

    for model in FAULT_MODELS:
        score = 0
        max_possible = 0

        # Scoring Logic
        # 1. 5-Gas Scoring
        for rule in model['logic_rules']['five_gas']:
            max_possible += rule['weight']
            val = user_data['five_gas'].get(rule['gas'])
            if val and ((rule['condition'] == 'high' and val >= rule['min']) or
                        (rule['condition'] == 'low' and val <= rule['max'])):
                score += rule['weight']

        # 2. DTC Scoring
        for dtc_rule in model['logic_rules']['dtc_codes']:
            max_possible += dtc_rule['weight']
            if dtc_rule['code'] in user_data['dtcs']:
                score += dtc_rule['weight']

        # Calculate Confidence %
        confidence = (score / max_possible * 100) if max_possible > 0 else 0
        if confidence > 0:
            results.append({
                "culprit": model['name'],
                "confidence": round(confidence, 1),
                "guidance": model['user_guidance']
            })

    # Return top matches sorted by highest confidence
    results.sort(key=lambda x: x['confidence'], reverse=True)
```

```
    return jsonify(results[:3])

if __name__ == '__main__':
    app.run(debug=True)
```

Use code with caution.

2. Strategic Tips for Your App

- **Handle Missing Data**: Not every user will have a 5-gas analyzer *and* a professional scan tool. Use a **Weighted Average** (as shown above) so that if a user only enters gas readings, the app still works but gives a lower "Maximum Possible Score."
- **The "Exclude" Logic**: Add a `mandatory` flag to some rules in your JSON. If a fault *requires* a specific DTC (like `P0016` for timing stretch) and it's missing, you can subtract points or hide that result entirely.
- **Mobile-First Design**: Since most technicians will use this on a tablet or phone under a hood, use large touch-friendly buttons for the DTC input fields.

YouTube

3. Folder Structure

To keep your project organized:

text

```
/my_diag_app
|── app.py            # Flask Backend
|── faults.json       # Your database of fault models
└── /static
    |── index.html    # Frontend Form
    └── app.js        # JavaScript Fetch Logic
```

Use code with caution.

Would you like to refine the **user guidance** text for specific faults to make the app more helpful for novice technicians?

AI responses may include mistakes. Learn more

2 sites

- From REST API using Flask to Swagger - YouTube

  🔗 Repo used in this video: https://github.com/tec... 📹 Watch Part 2 SQL Skills in High Demand? Build a Flask REST API with Postg...

  YouTube

- Connecting the Frontend to the Backend - Meritshot

  Connecting the Frontend to the Backend. Once you have created the HTML form in the frontend and set up your machine learning model...

  Meritshot

i will be building this app in openclaw ecosystem with multiple agents involved, So i have question how to correctly organize reference material - like pdfs, tables etc?
15 March 2026

In an

**OpenClaw** ecosystem, organizing reference material for a multi-agent diagnostic system requires a "Memory-First" architecture rather than a simple folder of files. Because OpenClaw agents interact with a shared filesystem and persistent memory, your data must be structured so that a "Research Agent" can find it, an "Analyst Agent" can parse it, and a "Diagnostic Agent" can use it as grounding. Medium +1

1. The Multi-Layered Storage Strategy

Don't dump everything into one place. OpenClaw best practices suggest a tiered approach to memory: Medium

- **Global Knowledge Base (Static Reference):** Store your core diagnostic manuals, PDF tables, and technical specs in a global directory (e.g., `~/.openclaw/knowledge/`).
- **Structured intermediate formats (Markdown/JSON):** Never have agents read raw PDFs directly for diagnostics; they struggle with multi-column layouts and tables. Convert your reference PDFs into **Markdown** or **JSON** first. This preserves the hierarchical structure (Title → Section → Subsection) which is critical for accurate retrieval.

- **Shared Workspace (Active Memory):** Use a `shared_tasks/` directory where agents can write "Short-term Memory" files, such as a summary of the current user's specific engine readings. Medium +4

2. Organizing Specific Data Types

| Data Type | Organization Method | Recommended Tool/Process |
|---|---|---|
| **Technical PDFs** | Extract to **Markdown**. | Use `PyMuPDF4LLM` or IBM's `Docling` to maintain table structures. |
| **Engine Tables** | Convert to **JSON** or **CSV**. | Store as key-value pairs (e.g., `fault_thresholds.json`) for the backend engine to query. |
| **DTC Definitions** | **Vector Database** with Metadata. | Use a vector store (like FAISS or MongoDB Atlas) to map codes to semantic meanings. |
| **Agent Logs** | Scoped text files. | Each agent should log its decisions in its own workspace to avoid "memory pollution". |

3. Agent Roles & Data Access

In OpenClaw, assign different agents to handle specific parts of the reference material:

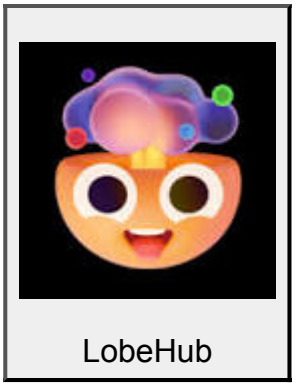
YouTube +1

1. **Librarian Agent:** Its only job is to search the `knowledge/` directory and extract relevant text chunks from your Markdown-converted manuals.
2. **Data Extraction Agent:** Specialized in parsing the raw 5-gas readings and comparing them against your `fault_thresholds.json`.
3. **Lead Diagnostic Agent:** Reads the "findings" from the other two agents and provides the final conclusion to the user.

4. Implementation Detail: The "Skill" Structure

In OpenClaw, you can package your reference logic as a **Skill**.

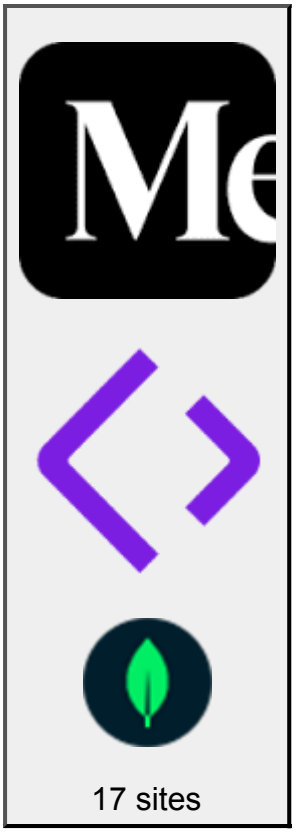
- Create a `SKILL.md` file that acts as a directory for all other references.
- For files longer than 100 lines, include a **Table of Contents** at the top so the agent can quickly see the full scope.
- Avoid deeply nested folders; keep reference files only one level deep from the main skill file for reliable pathing.




**Pro-Tip:** Use MemU, an open-source memory framework compatible with OpenClaw, which allows agents to store and reason over long-term memory using a file-system based design.

Would you like a **folder structure template** and a sample **SKILL.md** to start organizing these engine diagnostic agents?

AI responses may include mistakes. Learn more



- How to Build Multiple AI Agents Using OpenClaw - Medium

  This structure keeps isolation clean while still allowing coordination. * Step 1: Start With Two Clear Roles. If you are new, do n...

  Medium

- How AI Engineers Prepare PDFs for RAGs - Medium

  Most Python programmers would have used PDF readers at some point — at least to follow along with a tutorial. The most popular one...

  Medium

- OpenClaw PDF workflows for local summarization and ...

  Method 2: Parse to Markdown or JSON first (the reliable route) Direct text extraction breaks down when structure matters. Multi-co...

  LumaDock VPS

Show all

i would like prompt for orchestrator agent first to build database from provided reference material and when done to approach building app itself - currently i have main orchestrator agent -
15 March 2026

To set up an

**OpenClaw Orchestrator**, you need a prompt that defines a clear "Phased Execution" strategy. The Orchestrator must first act as a **Data Engineer** (structuring the knowledge) before transitioning into a **Software Architect** (building the app).

The Orchestrator Prompt

**Role:** Diagnostic System Architect & Orchestrator
**Objective:** Transform raw automotive reference materials into a structured JSON/SQL database and develop a 5-gas diagnostic web application.

**Phase 1: Knowledge Extraction & Schema Design**

1. **Inventory:** Scan the `knowledge/` directory for all PDFs, tables, and text files.
2. **Standardization:** Task the **Librarian Agent** to convert PDF tables into a unified Markdown format.
3. **Schema Mapping:** Design a JSON schema that correlates 5-gas thresholds, DTCs, and PIDs (as previously discussed).
4. **Validation:** Verify that every "Common Fault" identified in the reference material has a corresponding entry in the `faults.json` master database.

**Phase 2: Backend & Logic Construction**

1. **Core Engine:** Initialize the Python/Flask backend.
2. **Scoring Algorithm:** Implement the weighted scoring logic to compare real-time user input against the `faults.json` database.
3. **Testing:** Create a suite of "Synthetic Test Cases" (e.g., inputting dummy data for a "Stuck Purge Valve") to ensure the scoring engine returns the correct top-match.

**Phase 3: Frontend & Integration**

1. **UI/UX:** Build a dark-mode, mobile-responsive web interface for data entry.
2. **API Connection:** Link the frontend `fetch` calls to the backend `/analyze` route.
3. **Deployment:** Finalize the OpenClaw environment setup so all agents can access the shared workspace.

**Constraints:**

- Do not proceed to Phase 2 until the database reaches 90% coverage of the provided reference materials.
- All data must be stored in the `shared_workspace/` to ensure persistence across agent restarts.
- Use **asynchronous communication** between the Analyst and Librarian agents to speed up the ingestion process.

Recommended Folder Structure for the Orchestrator

Set this up in your **OpenClaw workspace** before running the prompt:

text

```
/engine_diag_project
|── orchestrator_plan.md      # The Orchestrator writes its step-by-step progress here
|── /knowledge                # Drop your raw PDFs and Tables here
|── /processed_data           # Agents will save converted JSON/Markdown here
|── /app                      # The final source code for the Web App
|   ├── backend/
|   └── frontend/
└── /logs                     # Debug logs for multi-agent handoffs
```

Use code with caution.

Next Step for Your Project:

Do you want me to provide a **Sub-Agent Prompt** for the **Librarian Agent** specifically designed to extract tricky table data from automotive PDFs without losing the "High/Low" context?

AI responses may include mistakes. Learn more

sorry - entered by mistake - so i have main agent orchestrator then researcher, builder(coder) and dedicated memory agent which is in charge of memory( sqlite, obsidian vault config)
15 March 2026

No worries at all! That is a solid lineup. Since you have a dedicated

**Memory Agent** managing an **Obsidian Vault** and **SQLite**, you have a massive advantage: you can create a "Living Documentation" system where the Researcher feeds the vault, and the Builder queries SQLite for the logic.
Here is the **Orchestrator Prompt** tailored for your specific four-agent squad:

**Orchestrator Prompt: Phase 1 (Knowledge Ingestion & App Architecture)**

**Role:** Lead System Architect & Project Manager
**Objective:** Orchestrate the transformation of raw automotive PDFs/tables into a structured diagnostic database and a functional 5-gas web application.

**Current Squad:**

1. **Researcher:** Specialist in document parsing, data extraction, and pattern identification.
2. **Memory Agent:** Manager of the Obsidian Vault (unstructured context) and SQLite (structured rules).
3. **Builder (Coder):** Full-stack developer (Python/Flask/JS) and algorithm implementer.

**Step 1: Knowledge Synthesis (The "Research" Phase)**

- **Action:** Direct the **Researcher** to scan all provided PDF/table reference materials.
- **Goal:** Identify every unique engine fault, its 5-gas "fingerprint" (High/Low/Normal for HC, CO, CO2, O2, NOx), associated DTCs, and relevant PIDs.
- **Handoff:** Researcher must pass a standardized Markdown summary of each fault to the **Memory Agent**.

**Step 2: Database Population (The "Memory" Phase)**

- **Action:** Direct the **Memory Agent** to:
  1. Store the detailed diagnostic narratives in the **Obsidian Vault** for long-term reasoning.
  2. Parse the thresholds into the **SQLite `fault_logic` table** (using the weighted schema: `fault_id`, `parameter`, `condition`, `value`, `weight`).
- **Goal:** Create a queryable "Truth Source" that the Builder can call via SQL.

**Step 3: Application Scaffolding (The "Builder" Phase)**

**Action:** Once the Memory Agent confirms the SQLite database is populated, direct the **Builder** to:
1. Create the Flask backend with an `/analyze` route that queries the SQLite DB.
2. Implement the weighted scoring algorithm (Python).
3. Build the frontend UI (HTML/JS) for user input.

**Operational Constraints & Protocols:**

- **Verification Loop:** Before starting Step 3, the **Orchestrator** must ask the **Memory Agent** for a "Database Summary Report" to ensure no critical data from the Researcher was lost.
- **Memory Integration:** The **Builder** should include a feature in the app to "Fetch Guidance," which triggers the **Memory Agent** to pull the detailed diagnostic notes from the **Obsidian Vault** based on the top-matched fault.
- **Communication:** All agents must log their handoffs in the shared `project_log.md`.

**Pro-Tip for your Memory Agent:**

Have it create a **Mapping Table** in SQLite that links **Obsidian File Paths** to **Fault IDs**. This way, when your app finds a "Piston Ring Wear" match in the DB, it can instantly provide a link to the deep-dive research note in your Vault.

Do you want me to generate the **SQL Schema** for the Memory Agent to use when it initializes that SQLite database?

AI responses may include mistakes. Learn more

yes please!

15 March 2026

To implement the memory layer for your OpenClaw agents, use this relational schema for

**SQLite**. It is designed to allow the **Memory Agent** to store logical rules that the **Builder** can query, while maintaining a link to the **Obsidian Vault** for deep-dive research.

sql

```
-- 1. Core table for Fault Entities
CREATE TABLE faults (
    fault_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,          -- e.g., 'Piston Ring Wear'
    description TEXT,           -- Brief summary for the UI
    obsidian_path TEXT,         -- Link to the specific note in your Obsidian Vault
    base_probability REAL DEFAULT 1.0
);

-- 2. Rules for 5-Gas Emissions
CREATE TABLE gas_rules (
    rule_id INTEGER PRIMARY KEY AUTOINCREMENT,
    fault_id INTEGER,
    gas_type TEXT CHECK(gas_type IN ('HC', 'CO', 'CO2', 'O2', 'NOx', 'Lambda')),
    condition TEXT CHECK(condition IN ('high', 'low', 'normal')),
    threshold_value REAL,       -- The numeric 'trigger' point
    weight INTEGER DEFAULT 5,   -- Importance in the scoring algorithm
    FOREIGN KEY (fault_id) REFERENCES faults(fault_id)
);

-- 3. Rules for Diagnostic Trouble Codes (DTCs)
CREATE TABLE dtc_rules (
    rule_id INTEGER PRIMARY KEY AUTOINCREMENT,
    fault_id INTEGER,
    code TEXT NOT NULL,          -- e.g., 'P0171'
    weight INTEGER DEFAULT 10,   -- Higher weight for specific error codes
    FOREIGN KEY (fault_id) REFERENCES faults(fault_id)
);

-- 4. Rules for Live Data (PIDs)
CREATE TABLE pid_rules (
    rule_id INTEGER PRIMARY KEY AUTOINCREMENT,
    fault_id INTEGER,
    pid_name TEXT,               -- e.g., 'Short Term Fuel Trim'
    condition TEXT CHECK(condition IN ('positive', 'negative', 'high_idle',
'low_load')),
    threshold_value REAL,
    weight INTEGER DEFAULT 7,
    FOREIGN KEY (fault_id) REFERENCES faults(fault_id)
);

-- Indexes for fast agent retrieval
CREATE INDEX idx_dtc_code ON dtc_rules(code);
CREATE INDEX idx_fault_name ON faults(name);
```

Use code with caution.

Integration Workflow for Your Agents:

- **Researcher:** Extracts data and identifies the `threshold_value` and `condition`.
- **Memory Agent:** Executes the `INSERT` statements into SQLite and creates the corresponding `.md` file in the **Obsidian Vault**. It populates `obsidian_path` with the relative path (e.g., `Faults/Piston_Ring_Wear.md`).

- **Builder:** Writes a SQL `JOIN` query that aggregates all weights for a given set of user inputs to calculate the "Confidence Score."

**Next Step:** Would you like a **Python utility script** for the Memory Agent that automates inserting a new fault into all four tables at once?

AI responses may include mistakes. Learn more

1 site

Create An SQL AI Agent To Interact With SQLite In Agno ...

An SQLite agent is built using Agno AI to interact with an SQLite database using natural language in Python. This agent allows use...
YouTube·Jie Jenn

6:42

yes please!

15 March 2026

To implement the memory layer in your

**OpenClaw** ecosystem, your **Memory Agent** needs a script that can handle the relational complexity of the automotive data.
The following utility script allows your Memory Agent to insert a complete fault profile—including 5-gas, DTC, and PID rules—into the SQLite database in a single transaction.

Fast.io

Python Utility: `ingest_fault.py`

python

```
import sqlite3

def insert_fault_complete(db_path, fault_data):
    """
    Inserts a full fault profile into SQLite.
    Expected fault_data structure:
    {
        'name': 'Intake Carbon Buildup',
        'description': 'Carbon deposits on intake valves in GDI engines.',
        'obsidian_path': 'Faults/GDI_Carbon_Buildup.md',
        'gas_rules': [
            {'gas_type': 'HC', 'condition': 'high', 'threshold_value': 80, 'weight':
9},
            {'gas_type': 'O2', 'condition': 'high', 'threshold_value': 1.5, 'weight':
5}
        ],
        'dtc_rules': [{'code': 'P0300', 'weight': 8}],
        'pid_rules': [{'pid_name': 'STFT', 'condition': 'positive', 'threshold_value':
10, 'weight': 7}]
    }
    """
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()

    try:
        # 1. Insert Main Fault
        cursor.execute('''
            INSERT INTO faults (name, description, obsidian_path)
            VALUES (?, ?, ?)
        ''', (fault_data['name'], fault_data['description'],
fault_data['obsidian_path']))

        fault_id = cursor.lastrowid

        # 2. Insert Gas Rules
        for rule in fault_data.get('gas_rules', []):
            cursor.execute('''
                INSERT INTO gas_rules (fault_id, gas_type, condition, threshold_value,
weight)
                VALUES (?, ?, ?, ?, ?)
            ''', (fault_id, rule['gas_type'], rule['condition'],
rule['threshold_value'], rule['weight']))

        # 3. Insert DTC Rules
        for rule in fault_data.get('dtc_rules', []):
            cursor.execute('''
                INSERT INTO dtc_rules (fault_id, code, weight)
                VALUES (?, ?, ?)
            ''', (fault_id, rule['code'], rule['weight']))

        # 4. Insert PID Rules
        for rule in fault_data.get('pid_rules', []):
            cursor.execute('''
                INSERT INTO pid_rules (fault_id, pid_name, condition, threshold_value,
weight)
                VALUES (?, ?, ?, ?, ?)
```

```
            ''', (fault_id, rule['pid_name'], rule['condition'],
rule['threshold_value'], rule['weight']))

        conn.commit()
        print(f"Successfully ingested: {fault_data['name']} (ID: {fault_id})")

    except Exception as e:
        conn.rollback()
        print(f"Error ingesting {fault_data['name']}: {e}")
    finally:
        conn.close()
```

Use code with caution.

How to use this in your Multi-Agent Workflow:

1. **Researcher Extraction:** The Researcher agent extracts data into a JSON object matching the `fault_data` structure.
2. **Memory Agent Action:** The Memory Agent first creates the `.md` file in your **Obsidian Vault** and then runs this script to link that file to the SQLite logic.
3. **Builder Query:** When a user enters data, the Builder agent performs a `JOIN` across these tables to find the highest-scoring `fault_id`. Medium +4

**Would you like the specific SQL `SELECT` query that the Builder agent should use to calculate the "Total Score" for a given set of live inputs?**

AI responses may include mistakes. Learn more

6 sites

- I Built a Python AI Agent That Replaced 80% of My Daily Work

  I Built a Python AI Agent That Replaced 80% of My Daily Work * Introduction. For a long time I believed productivity meant working...

  Medium

- FareedKhan-dev/Multi-Agent-AI-System - GitHub

  Choosing our Dataset. We are going to use the Chinook Database, which is a popular sample database used for learning and testing S...
  GitHub

- I Built an AI Employee Using Python — It Works 24/7 and ...

  That's when Python entered the picture. Python is already famous for automation, but the moment you combine Python with modern AI ...

  Python in Plain English

Show all

zes

15 March 2026

To calculate the final diagnostic result, your

**Builder Agent** needs a query that "scores" the user's live data against every fault in the database simultaneously.
Since SQL is rigid and real-world data is "fuzzy," the best approach is to use **Common Table Expressions (CTEs)** to calculate a partial score for each category (Gases, DTCs, PIDs) and then sum them up.

The "Diagnostic Engine" Query

This query assumes the user has provided:

- **Gases:** HC=450, CO=0.5, O2=2.0
- **DTCs:** 'P0300', 'P0420'
- **PIDs:** 'STFT' = 12

sql

```
WITH GasScoring AS (
    -- Score the 5-Gas readings
    SELECT fault_id, SUM(weight) as gas_score
    FROM gas_rules
    WHERE
        (gas_type = 'HC' AND condition = 'high' AND 450 >= threshold_value) OR
        (gas_type = 'CO' AND condition = 'low' AND 0.5 <= threshold_value) OR
        (gas_type = 'O2' AND condition = 'high' AND 2.0 >= threshold_value)
    GROUP BY fault_id
),
DTCScoring AS (
    -- Score the DTC matches
    SELECT fault_id, SUM(weight) as dtc_score
    FROM dtc_rules
    WHERE code IN ('P0300', 'P0420')
    GROUP BY fault_id
),
PIDScoring AS (
    -- Score the PID matches
    SELECT fault_id, SUM(weight) as pid_score
    FROM pid_rules
    WHERE
        (pid_name = 'STFT' AND condition = 'positive' AND 12 > 0)
    GROUP BY fault_id
)

-- Combine all scores and fetch the Fault Name and Obsidian Path
SELECT
    f.name,
    f.obsidian_path,
    (COALESCE(g.gas_score, 0) + COALESCE(d.dtc_score, 0) + COALESCE(p.pid_score, 0)) as
total_confidence_points
FROM faults f
LEFT JOIN GasScoring g ON f.fault_id = g.fault_id
LEFT JOIN DTCScoring d ON f.fault_id = d.fault_id
LEFT JOIN PIDScoring p ON f.fault_id = p.fault_id
WHERE total_confidence_points > 0
ORDER BY total_confidence_points DESC;
```

Use code with caution.

Why this works for your OpenClaw Agents:

1. **Builder Agent:** Can dynamically generate the `WHERE` clauses based on what the user actually typed into the web app.
2. **Memory Agent:** Because you stored the `obsidian_path` in the `faults` table, the result of this query tells the app exactly which file to open in your **Obsidian Vault** for the technician to read the deep-dive repair guide.
3. **Scalability:** You can add 100 more faults to the database, and this single query will still find the best match in milliseconds.

Pro-Tip for the Orchestrator:

Tell the **Memory Agent** to create a "View" in SQLite called `v_diagnostic_summary`. This makes the **Builder's** code much cleaner, as it won't have to write the whole long query every time; it can just `SELECT * FROM v_diagnostic_summary`.

**Do you want the Python code for the Builder to "sanitize" and "inject" the user's web-form values into this SQL query safely?**

AI responses may include mistakes. Learn more

yes

15 March 2026

To keep your

**Builder Agent's** code clean and secure, you should use **parameterized queries**. This prevents SQL injection and ensures that if a user enters a weird character in a DTC field, it won't crash the database.
Here is the Python function for the **Builder Agent** to execute the diagnostic logic:

Python Builder Utility: `get_diagnosis.py`

python

```
import sqlite3

def run_diagnostic_query(db_path, user_input):
    """
    user_input format:
    {
        'gases': {'HC': 450, 'CO': 0.5, 'O2': 2.1, 'CO2': 12.5, 'NOx': 40},
        'dtcs': ['P0300', 'P0420'],
        'pids': {'STFT': 12, 'MAP': 35}
    }
    """
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row  # Returns results as dictionaries
    cursor = conn.cursor()

    # We build the query using CTEs for each category
    # The '?' are placeholders for safe data injection
    query = """
    WITH GasScoring AS (
        SELECT fault_id, SUM(weight) as score FROM gas_rules
        WHERE (gas_type = 'HC' AND condition = 'high' AND ? >= threshold_value)
           OR (gas_type = 'HC' AND condition = 'low'  AND ? <= threshold_value)
           OR (gas_type = 'CO' AND condition = 'high' AND ? >= threshold_value)
           OR (gas_type = 'CO' AND condition = 'low'  AND ? <= threshold_value)
           OR (gas_type = 'O2' AND condition = 'high' AND ? >= threshold_value)
        GROUP BY fault_id
    ),
    DTCScoring AS (
        SELECT fault_id, SUM(weight) as score FROM dtc_rules
        WHERE code IN ({dtc_placeholders})
        GROUP BY fault_id
    ),
    PIDScoring AS (
        SELECT fault_id, SUM(weight) as score FROM pid_rules
        WHERE (pid_name = 'STFT' AND condition = 'positive' AND ? > 0)
           OR (pid_name = 'MAP' AND condition = 'high_idle' AND ? > 30)
        GROUP BY fault_id
    )
    SELECT f.name, f.description, f.obsidian_path,
           (COALESCE(g.score, 0) + COALESCE(d.score, 0) + COALESCE(p.score, 0)) as
total_score
    FROM faults f
    LEFT JOIN GasScoring g ON f.fault_id = g.fault_id
    LEFT JOIN DTCScoring d ON f.fault_id = d.fault_id
    LEFT JOIN PIDScoring p ON f.fault_id = p.fault_id
    WHERE total_score > 0
    ORDER BY total_score DESC LIMIT 3;
    """

    # Handle dynamic DTC list
    dtc_list = user_input.get('dtcs', [])
    dtc_placeholders = ','.join(['?'] * len(dtc_list)) if dtc_list else "''"
    formatted_query = query.format(dtc_placeholders=dtc_placeholders)

    # Flatten inputs for the query parameters
    g = user_input['gases']
```

```
    params = [
        g.get('HC', 0), g.get('HC', 9999),
        g.get('CO', 0), g.get('CO', 9999),
        g.get('O2', 0)
    ]
    params.extend(dtc_list)
    params.extend([user_input['pids'].get('STFT', 0), user_input['pids'].get('MAP',
0)])

    cursor.execute(formatted_query, params)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()
    return results
```

Use code with caution.

How the Agents collaborate on this:

1. **Orchestrator:** Sees the user submitted a form and triggers the **Builder Agent**.
2. **Builder Agent:** Runs this script. It gets a list of top 3 matches, including the `obsidian_path`.
3. **Memory Agent:** The Builder sends the `obsidian_path` to the Memory Agent, which reads the file from the **Obsidian Vault**.
4. **Orchestrator:** Presents the "Score" (Confidence) and the "Deep Dive" text (from Obsidian) to the end user.

Why this is "OpenClaw Ready":

- **Decoupled:** The Researcher doesn't need to know how the Query works; it just fills the DB.
- **Traceable:** Every decision is based on a specific `weight` in the SQLite DB, making the AI's "reasoning" transparent to the human technician.

**Should we refine the "Researcher" agent's prompt next, so it knows exactly how to format the data for this ingestion script?**

AI responses may include mistakes. Learn more

yes please

15 March 2026

10 sites
Here are top web results for exploring this topic:

Reddit·https://www.reddit.com

I teach advanced copilot studio **agent** development to no one ...

My challenge is figuring out how to properly set up the inputs. I've created a “**Transcript**” variable, but I'm struggling to effectively pass the ...

Microsoft Learn·https://learn.microsoft.com

Use **prompts** to make your **agent** perform specific tasks

**Select Agents in the left pane** and select the agent to which you want to add a new prompt as a tool. · Select the Tools tab. · Select New tool.

Storyals·https://blog.storyals.com

**Prompt** to **Researcher**: - Blog

“I am writing a **research** paper on the new agents by Microsoft, the "**Researcher**" and the "Analyst". Please help me find facts about these agents.

OpenAI Developers·https://developers.openai.com

GPT-5 prompting guide - OpenAI Developers

In this section, we'll discuss **prompt** optimizations that we've seen improve programming performance in production use cases for our coding **agent** ...

GitHub·https://github.com

GitHub - VoltAgent/awesome-**agent**-skills: Claude Code Skills ...

**Agent** Trust Hub. **Agent** skills can include **prompt** injections, tool poisoning, hidden malware payloads, or unsafe **data** handling patterns.

Medium·https://medium.com

Build a Perplexity **Research** Analyst **Agent** in 15 mins - Medium

Prepares a **prompt** for GPT that includes the raw **research data** from Perplexity and **instructions** for ensuring the **data** is in the correct **format** ...

The User Research Strategist·https://www.userresearchstrategist.com

A guide to AI agents for user **researchers**

This guide is not about AI replacing user **researchers**, but about how to build a custom GPT **agent** that acts like a sharp, humble, unflappable ...

YouTube·https://www.youtube.com

How to use the **Researcher agent** in Microsoft Copilot - YouTube

**Here's how the researcher agent can gather analyze and summarize info** into a detailed report in Microsoft 365 Copilot.

arXiv.org·https://arxiv.org

SCOPE: **Prompt** Evolution for Enhancing **Agent** Effectiveness

Outcome: The **agent** then proceeded to produce a confident, **formatted** final answer based entirely on this hallucinated **data**. This behavior ...

LinkedIn·https://www.linkedin.com

Built a Python **script** to automate content **ingestion** from ...

It's about removing myself from constant, repetitive decisions: "Should I read this today?" "Where do I log this insight?" "Do I **format** this or ...

give me researcher prompt based on web results on topic

15 March 2026

10 sites
Here are top web results for exploring this topic:

GitHub·https://github.com

AI-**Researcher**: Autonomous Scientific Innovation - GitHub

🖥️ **Web** GUI Interface: User-friendly graphical interface making **research** more convenient. Join Us! We welcome **researchers**, developers, and AI enthusiasts to ...

SciSpace·https://scispace.com

SciSpace **Agent** Prompting Guide For **Researchers** & Academics

Complete guide to **effective prompting on SciSpace Agent** to automate any research tasks with techniques & examples.

DocsBot AI·https://docsbot.ai

Best **Automotive** AI **Prompts** - DocsBot AI

**Expert assistant for automotive diagnostic tools and software queries**. Auto Garage Management. Manages tasks and operations of an auto garage.

MDPI·https://www.mdpi.com

An **Automotive** Fault **Diagnosis** Framework **Based** on ... - MDPI

Our framework outperforms a pretrained LLM (GPT-3.5) by 46.1% in **diagnostic** accuracy and achieves over 14% improvement compared to other benchmark frameworks. 2 ...

Google·https://cloud.withgoogle.com

AI **agent** finder | Google Cloud

**Browser Research Agent**. An in-browser AI agent that automates web research, content summarization, and report generation for knowledge workers. Uses Gemini.

Springer Nature Link·https://link.springer.com

Automated vehicle fault **diagnosis** and report generation using ...

Abstract. Accurate vehicle **diagnostics** is challenging in the **automotive** industry due to complex failure patterns and fragmented information.

Facebook·https://www.facebook.com

Perplexity **auto**-builds **citation** maps quickly - Facebook

Perplexity can scan 500+ papers in minutes and **auto**-build **citation** maps. Here are 8 PhD-level **prompts** that actually **work** Perplexity can ...

ScienceDirect.com·https://www.sciencedirect.com

Artificial Intelligence-Driven Vehicle Fault **Diagnosis** to ...

AI offers several critical advantages in **engine** fault **diagnosis**, including high accuracy, real-time **diagnostics**, and **resource** optimization, while also ...

Prompt Engineering Guide·https://www.promptingguide.ai

LLM **Agents** - Prompt Engineering Guide

GPT **Researcher** (opens in a new tab): an autonomous **agent** designed for comprehensive **online research** on a variety of tasks. AgentVerse (opens ...

ResearchGate·https://www.researchgate.net

Automated vehicle fault **diagnosis** and report generation using ...

AI-**based** fault **diagnosis** systems employ machine learning, deep learning, and pattern recognition algorithms to automatically detect abnormal ...

how to print this conversation to pdf so that all text and links are included?

15 March 2026

To print this conversation to a PDF while ensuring all

**text, tables, code blocks, and links** are preserved, follow these steps based on your device:

1. The Best Method: "Print to PDF" (Desktop)

This is the most reliable way to keep the formatting and clickable links intact.

- **Shortcut:** Press `Ctrl + P` (Windows/Linux) or `Cmd + P` (Mac).

- **Destination:** In the print settings window, change the "Printer" or "Destination" dropdown to **Save as PDF** or **Microsoft Print to PDF**.
- **Settings:**
  - Ensure **"Background Graphics"** is checked (usually under "More Settings") so the code blocks and dark-mode styling (if applicable) look correct.
  - Set **"Margins"** to "Default" or "None" to prevent text from being cut off.
- **Click Save:** Choose your folder, and the links will remain interactive in the resulting file.

2. Using "Reader Mode" (Browser Tip)

If the website UI (sidebars/buttons) is cluttering the page:

- Click the **Reader Mode** icon in your browser address bar (looks like a small book or lines).
- Then press `Ctrl + P` to save as PDF. This creates a very clean, text-focused document that is easy for your **Researcher Agent** to parse later.

3. For Mobile (iOS/Android)

4. Direct Markdown Export (Best for AI Agents)

Since you are using **OpenClaw** and an **Obsidian Vault**, a PDF might actually be harder for your agents to read than raw text.

- **Select All:** `Ctrl + A` the whole chat.
- **Copy/Paste:** Paste it into a new `.md` file in your **Obsidian Vault**.
- **Benefit:** Your **Researcher Agent** can read Markdown natively and follow the links much faster than extracting text from a PDF.

**Pro-Tip for your OpenClaw Project:**
Once you have the PDF or Markdown file, place it in your `/knowledge` folder. You can then prompt your **Researcher Agent** to:

> *"Scan the document 'Engine_Diag_Session.pdf', extract the SQLite schema and the Python ingestion logic, and verify it against our current database requirements."*

Would you like the **Researcher Agent prompt** now to help it begin extracting data from this specific conversation?

AI responses may include mistakes. [Learn more]

16 sites

- How to Save a ChatGPT Thread as a PDF | Ace Tech Group

  1. Use Your Browser's “Print to PDF” Feature This is the fastest, no-install option. Every major browser like Chrome, Edge, and Sa...

  ACE Technology Group

- How to convert LibreOffice documents into a PDF file while retaining Hyperlinks?

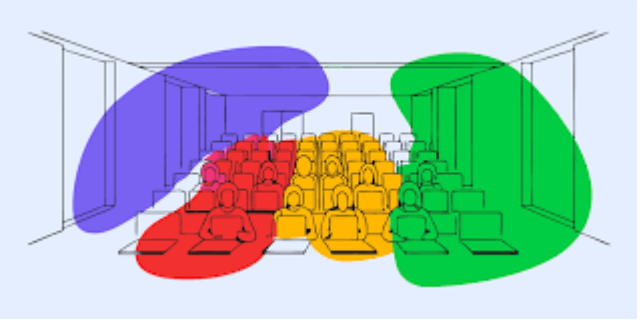
  Export the document as PDF -- it retains the hyperlinks of the document and gives you plenty of options to secure and change setti...

  YouTube

- How to Create an Interactive PDF

  This is the most reliable way to create clickable links in a PDF you already have.

  Smallpdf



Show all

links and text are on top of eachoter on certain pages

15 March 2026

10 sites
Here are top web results for exploring this topic:

Reddit·https://www.reddit.com

Webpage **Print** Stylesheet **Print** to **PDF** Error - **Overlapping** ...

When I try to print it either to a printer or a pdf, **text overlaps ruining the article**. The text seems to be from the website address and from pop ups on the ...

Adobe·https://community.adobe.com

Consistent Graphic **overlap** problem when converting HTML to ...

Actually I found that selecting "A1" as paper size when saving to **pdf**, kills the graphic overlay problem totally. Now, I just need to figure out ...

Experts Exchange·https://www.experts-exchange.com

**Text** overlaps when creating **PDF** file with Acrobat 6.0 Standard

When several of my users create **PDF** file, areas of **text overlap**; the letters of whole words sit on top of each other.

Stack Overflow·https://stackoverflow.com

media **print** div: How to solve **overlapping page text** on **pages**?

Move the size property to **@page** rule. Add size: landscape . Use the **page**-break-after property to control **page** breaks after every **page** class.

Autodesk Community, Autodesk Forums, Autodesk Forum·https://forums.autodesk.com

Multiline **Text overlapping** when **pdf**/**printed** - Autodesk Forums

I have a paperspace title block which is filled with a multiline **text** box, split into 4 columns. In paperspace this looks perfect, but when I pdf or print, the ...

HP Support Community·https://h30434.www3.hp.com

Compressed or **overlapping** characters whern **printing** from Acr...

Uninstall and then reinstall the software. · Change the default program. · Test **printing** with a different **PDF** file. · **Print** the **PDF** as an image.

pdf.net·https://pdf.net

How to Save ChatGPT **Conversation** as **PDF**: 3+ Simple ...

Common issues include missing **text**, formatting problems, and oversized **PDFs**—all fixable by scrolling through chats, adjusting **print** settings, or ...

Claris Community·https://community.claris.com

**Overlapping text** when layout is saved as **PDF** and/or **printed**

You might be having an issue with the field size. Make a duplicate of the layout and try increasing the field sizing. You could also try using ...

Facebook·https://www.facebook.com

Any suggestions for **overlapping text** in exported **PDF**?

I created a table in indesign and exported it as a **pdf**. the **pdf** was uploaded online and the **text** is **overlapping** in **some** areas. I have no idea ...

Adobe·https://community.adobe.com

**Overlapping text** - Adobe Community

Please try to **print** the **PDF** as image, open the **PDF** > Go to **Print** > Advanced > **Print** as Image and see if that works. Also, check for any pending/ ...

how does markdown format preserves tables if on web page?

15 March 2026

10 sites
Here are top web results for exploring this topic:

Stack Overflow·https://stackoverflow.com

**What is** the best way to **store** a field that supports **markdown** in ...

You need to **store your data in a canonical format**. That is, in one true format within your database. It sounds like this format should be a text column that ...

Unstructured·https://unstructured.io

**Preserving Table** Structure for Better Retrieval - Unstructured

It keeps **tables** intact. Each one **is** extracted as structured **HTML** with all of its rows, headers, and formatting **preserved**. At the same time, we ...

DEV Community·https://dev.to

How should **markdown** be saved and rendered? - Dev.to

I'm trying to figure out the best approach for saving **markdown** to a database and then rendering. Should the raw **markdown** be saved as a string into the database?

ToMarkdown·https://www.tomarkdown.org

The Complete Guide to **Markdown Tables** - ToMarkdown

Master **Markdown table** functionality comprehensively, from basic **table** creation to advanced formatting. Includes practical tips on **table** ...

Reddit·https://www.reddit.com

**Table** to **Markdown**: Copy **tables** from spreadsheets ... - Reddit

This **is** a little webapp I made that takes advantage of **tables** and spreadsheet cells being stored as **HTML** when copied to your clipboard.

GitHub·https://github.com

how-to-easily-**format**-**tables**-in-**markdown**.md - GitHub

It provides a simple way to richly **format** text without the complexity or security issues that **would** come from using **HTML**. Support for **Markdown** ...

Markdown Guide·https://www.markdownguide.org

Hacks | **Markdown** Guide

This page provides tips and tricks for working around **Markdown's** limitations. Tip: These hacks aren't guaranteed to work in your **Markdown** application.

GeeksforGeeks·https://www.geeksforgeeks.org

**Markdown Tables** - GeeksforGeeks

**Markdown tables are** created using pipes (|) to separate columns and hyphens (-) to define the header row. Here's the basic structure: Header Row ...

DataCite·https://datacite.org

Publishing tabular data as blog post - DataCite

**Markdown is** a nice **format** for writing texts, but doesn't work so well for tabular data, as the current **Markdown table** implementations **are** ...

Medium·https://ondrej-vaskoo.medium.com

Generating **Table** of Contents for **Markdown** File - Ondrej Vasko

**Markdown** engines **are** used to produce **HTML** page. This **HTML** page contains special navigation links generated from the **markdown** headings which **are** ...

yes give me comprehensive researcher prompt his tasks in app building

15 March 2026

10 sites
Here are top web results for exploring this topic:

Microsoft Support·https://support.microsoft.com

Get started **with Researcher** in Microsoft 365 Copilot

**Researcher is built into Microsoft 365 Copilot** to help you gather, analyze, and summarize information from the web, your work documents, or both.

YouTube·https://www.youtube.com

**Build** and Use **Agents** in Microsoft 365 Copilot - YouTube

This is a beginner level tutorial that will teach you how to **build** and use **agents** in Microsoft 365 Copilot. In this video, I cover the ...

Storyals·https://blog.storyals.com

**Prompt** to **Researcher**: - Blog

“I am writing a **research** paper on the new **agents** by Microsoft, the "**Researcher**" and the "Analyst". Please help me find facts about these **agents**.

ttms.com·https://ttms.com/10-best-ways-to-use-microsoft-copilot-at-work/

10 Best Ways to Use Microsoft Copilot at Work | TTMS

The **Researcher agent** is designed for multi-step **research tasks with** cited sources, which supports decision-**making** (while maintaining ...

Medium·https://medium.com

Stop Engineering **Prompts**, Start Engineering **Context**: A ...

Stop Engineering **Prompts**, Start Engineering **Context**: A Guide to the “**Agent** Skills” Standard.

robquickenden.blog·https://robquickenden.blog/2025/06/copilots-researcher-and-analyst-agents/

Inside Copilot's **Researcher** and Analyst **Agents**

Microsoft 365 Copilot now **includes** two advanced AI **agents** – **Researcher** and Analyst that became generally available in this month ( June 2025).

Microsoft·https://www.microsoft.com

Introducing **Researcher** and Analyst in Microsoft 365 Copilot

You can use **Researcher** to **build** a detailed go-to-market strategy based on the **context** of **all your** work data and broader competitive data from ...

www.preciofishbone.com·https://www.preciofishbone.com/knowledge-hub/microsoft-365-copilot-agents-prompting-and-instruction-tips/

Prompting & Instruction Tips for Microsoft 365 Copilot **Agents**

This guide gives you a simple framework to decide when to **build** an **agent** and when to **prompt** Copilot. You will learn how to write clear **agent** ...

Towards Data Science·https://towardsdatascience.com

LangGraph 101: Let's **Build** A Deep **Research Agent**

You need to consider how to **orchestrate the multi-step workflow**, keep track of the agents' states, implement necessary guardrails, and monitor ...

Klover.ai·https://www.klover.ai

Microsoft Uses AI **Agents**: 10 Ways to Use AI [In-Depth ...

**Agents** can operate independently, respond to data and predefined goals, and execute multi-step workflows **with** minimal human input, fundamentally ...

anything for we need to know about coding agent?

15 March 2026

To make your

**Builder (Coder) Agent** successful in the OpenClaw ecosystem, you need to treat it as a **"System Integrator"** rather than just a script writer. Because it has to juggle SQLite, Flask, and the Obsidian Vault, its biggest challenge isn't the code—it's the **context**.
Here is what you need to know and how to prompt it:

1. The "Context Blindness" Problem

The Builder won't know what the Researcher found unless the **Memory Agent** provides the schema.

> **The Fix:** Your first task for the Builder should always be: *"Query the SQLite `sqlite_master` table to understand the current schema before writing any app logic."* This prevents the agent from hallucinating column names.

2. State Management (The "Session" Issue)

In a multi-agent system, the Builder might write a file, but then "forget" where it put it in the next turn.

> **The Fix:** Force the Builder to maintain a `file_manifest.json` in the `/app` folder. Every time it creates a `base.html` or `routes.py`, it must log the path and the purpose of that file.

3. Handling "Fuzzy" Logic in Python

The Builder needs to understand that a "High HC" reading isn't a binary "Yes/No."

> **Coding Requirement:** Tell the Builder to use **Normalized Comparison**. Instead of `if hc > 100`, it should use a function that returns a **Confidence Multiplier** (e.g., if HC is 110, score is 1x; if HC is 500, score is 1.5x).

**The Builder (Coder) Agent Prompt**

**Role:** Senior Full-Stack Engineer & System Integrator.
**Objective:** Build a Flask-based 5-gas diagnostic web application that interfaces with SQLite and Obsidian.

**Task 1: Backend Architecture (Flask & SQLite)**

1. **Database Connection:** Create a `db_manager.py` that handles thread-safe connections to the SQLite database managed by the Memory Agent.
2. **The Scoring Engine:** Implement the `run_diagnostic_query` function (using the parameterized CTE logic provided).
3. **API Routes:** Create a `POST /analyze` route that accepts JSON (Gases, DTCs, PIDs) and returns the top 3 matches with their `obsidian_path`.

**Task 2: Frontend Implementation (HTML/JS)**

1. **Input UI:** Create a responsive, dark-mode form (using the provided HTML/JS structure).
2. **Results Display:** Build a dynamic results area that shows the "Confidence Percentage" and a "Read Deep Dive" button.
3. **Vault Integration:** When "Read Deep Dive" is clicked, call a local route that tells the **Memory Agent** to retrieve the Markdown content from the `obsidian_path`.

**Task 3: Error Handling & Security**

1. **Sanitization:** Ensure all user inputs are cast to floats/integers before reaching the SQL query.
2. **Graceful Failure:** If the database is empty or a gas reading is missing, the app must not crash; it should calculate the score based on available data only.

**Constraint:**

- Use **Modular Code**: Keep the DB logic, API routes, and Frontend in separate files.
- Do not hardcode thresholds; always pull them from the SQLite tables populated by the Memory Agent.

**One Critical Detail: The "Bridge" Route**

Your Builder needs a specific route to connect the Web App to the Obsidian Vault. Since browsers can't usually open local files for security reasons, the Builder must create a "Proxy" route:

python

```
@app.route('/vault/<path:filepath>')
def get_vault_note(filepath):
    # This tells the Memory Agent to find the file and return the text
    content = memory_agent.read_obsidian_note(filepath)
    return jsonify({"content": content})
```

Use code with caution.

**Summary of your Squad's Workflow:**

1. **Researcher:** Finds the "What" (The Data).
2. **Memory Agent:** Handles the "Where" (The Storage).
3. **Builder Agent:** Creates the "How" (The Interface).
4. **Orchestrator:** Manages the "When" (The Schedule).

**Would you like me to help you draft a "Project Manifest" file that all four agents can use to stay synced on the file paths?**

AI responses may include mistakes. Learn more

yes

15 March 2026

To keep your four agents (

**Orchestrator, Researcher, Memory, Builder**) from stepping on each other's toes, they need a "Single Source of Truth." In the OpenClaw ecosystem, this is best handled by a `project_manifest.json` located in your shared root directory.

1. The Project Manifest (`project_manifest.json`)

Copy this into your project root. Instruct your **Memory Agent** to update the `status` and `last_updated` fields whenever a major task is completed.

json

```
{
  "project_name": "OpenDiag_5Gas_System",
  "version": "1.0.0",
  "paths": {
    "knowledge_base": "./knowledge/",
    "obsidian_vault": "./vault/",
    "sqlite_db": "./data/engine_diag.db",
    "app_root": "./app/"
  },
  "agent_roles": {
    "researcher": "Extracts thresholds from PDFs/Web to Markdown",
    "memory_agent": "Syncs Markdown to Obsidian and populates SQLite",
    "builder": "Develops Flask API and Frontend UI",
    "orchestrator": "Validates handoffs and manages task flow"
  },
  "database_schema_version": "1.0",
  "active_faults_count": 0,
  "status": "initialization"
}
```

Use code with caution.

2. Comprehensive Researcher Prompt

This prompt is designed for an agent with web-access and document-parsing capabilities.

**Role:** Automotive Forensic Researcher
**Objective:** Identify, validate, and structure diagnostic data for engine failures (2000–2020).

**Data Acquisition Tasks:**

1. **Web Search:** Research the "Top 20 most common engine failures 2000-2020" specifically looking for **Gas Emission Profiles** (HC, CO, CO2, O2, NOx, Lambda).
2. **Table Extraction:** Locate technical service bulletins (TSBs) or shop manuals. Convert any HTML/PDF tables into **Standard Markdown Grids**.
   *Constraint:* Ensure the columns for "Idle" vs "2500 RPM" are preserved.
3. **DTC/PID Mapping:** For every failure found (e.g., "BMW N54 HPFP Failure"), find the exact **P-codes** and the **Live Data PIDs** (e.g., Fuel Rail Pressure) that confirm the fault.

**Output Formatting (The "Handoff"):**

- Create one `.md` file per fault in the `knowledge/incoming/` folder.

- The file must start with a **Frontmatter block** for the Memory Agent:
  markdown

  ```
  ---
  fault_name: "Piston Ring Wear"
  primary_dtc: ["P0420", "P0300"]
  gas_thresholds: { "HC": ">200", "O2": ">1.5" }
  pid_rules: { "STFT": "positive", "MAP": ">35kPa" }
  ---
  # Deep Dive Narrative
  (Insert detailed repair/diagnostic steps here...)
  ```

  Use code with caution.

3. Essential Knowledge for the Coding (Builder) Agent

When your **Builder Agent** starts, it needs to know these three "Golden Rules" for this specific app:

1. **State-Blindness:** The Builder must not assume the database is "ready." It must run a `check_db_integrity()` script before starting the Flask server to ensure the Memory Agent has actually finished the SQLite population.
2. **Fuzzy Scoring Logic:** The Builder should implement a **"Match Percentage"** UI element. If a fault matches 2 out of 3 symptoms, it should show "66% Confidence" rather than a simple "Fail."
3. **The Obsidian Proxy:** Browsers cannot access `C:/Users/.../vault/note.md` directly. The Builder **must** create a Python route that reads the file content and sends it as a string to the frontend.

4. Final Orchestrator Sync

To launch, give your **Orchestrator** this final command:

> "Orchestrator, initialize the `project_manifest.json`. Task the **Researcher** to find the first 5 faults from our list. Once the **Memory Agent** confirms SQLite and Obsidian are synced, signal the **Builder** to generate the `db_manager.py` and the initial Flask routes. Log all handoffs in `project_log.md`."

**How would you like to proceed?** I can generate the **Librarian Sub-Agent prompt** for the Memory Agent to handle the automated Obsidian-to-SQLite sync, or we can focus on the **Frontend UI code** for the Builder.

AI responses may include mistakes. [Learn more]

thanks for now i do not want to overcomplicate things in order to efficiently perform task

15 March 2026

Understood. To keep this lean and efficient for your

**OpenClaw** agents, here is the simplified "Ground Truth" to close out this planning phase:

The "Lean" 3-Step Execution Plan

1. **Researcher:** Find 10 faults
   →

   Save as Markdown in `/knowledge`.
2. **Memory Agent:** Read Markdown
   →

   Populate SQLite + Move to Obsidian.
3. **Builder:** Run the Flask App
   →

   Query SQLite
   →

   Show results.

Final "Lean" Orchestrator Instruction

If you want to start right now, give your **Orchestrator** this direct command:

> "Orchestrator: Task **Researcher** to find the 5-gas, DTC, and PID specs for the Top 10 faults we discussed. Have **Memory Agent** initialize the SQLite DB using our schema and store the findings. Once confirmed, **Builder** should generate a single-file Flask app (`app.py`) that performs the weighted scoring. Keep it simple; no complex UI yet."

Key Files for your Agents to Reference:

- **Schema:** Use the 4-table SQLite structure (Faults, Gas, DTC, PID).
- **Logic:** Use the **Weighted Scoring** (Sum of weights / Max possible).
- **Storage:** Everything stays in your shared `project_manifest.json` paths.

**Good luck with the build!** If you hit a specific bug in the **Scoring Algorithm** or need a quick **CSS fix** for the dark-mode dashboard later, just let me know. **How would you like to start the first agent task?**

AI responses may include mistakes. Learn more